

Эмуляция и измерение задержек в глобальных сетях

Лабораторная работа № 4

Шулуужук Айраана НПИбд-02-22

Содержание

1	Цель работы	6
2	Выполнение лабораторной работы	7
2.1	Запуск лабораторной топологии	7
2.2	Интерактивные эксперименты	8
2.3	Изменение задержки в эмулируемой глобальной сети	9
2.4	Восстановление исходных значений (удаление правил) задержки в эмулируемой глобальной сети	10
2.5	Добавление значения дрожания задержки в интерфейс подключения к эмулируемой глобальной сети	11
2.6	Добавление значения корреляции для джиттера и задержки в интерфейс подключения к эмулируемой глобальной сети	11
2.7	Распределение задержки в интерфейсе подключения к эмулируемой глобальной сети	13
2.8	Воспроизведение экспериментов. Предварительная подготовка	13
2.9	Добавление задержки для интерфейса, подключающегося к эмулируемой глобальной сети	14
2.10	Задание для самостоятельной работы	20
3	Выводы	27

Список иллюстраций

2.1	запуск виртуальной машины и исправление прав доступа .	7
2.2	запуск графических окон	8
2.3	проверка подключения между хостами	8
2.4	проверка подключения соединения с задержкой 100 мс . . .	9
2.5	проверка подключения соединения с задержкой 100 мс на хосте 2	9
2.6	Изменение задержки в эмулируемой глобальной сети со 100 мс до 50 мс	10
2.7	Восстановление исходных значений (удаление правил) задержки в эмулируемой глобальной сети	10
2.8	Добавление значения дрожания задержки в интерфейс подключения к эмулируемой глобальной сети	11
2.9	Добавление значения корреляции для джиттера и задержки в интерфейс подключения к эмулируемой глобальной сети	12
2.10	Добавление значения корреляции для джиттера и задержки в интерфейс подключения к эмулируемой глобальной сети	12
2.11	Распределение задержки в интерфейсе подключения к эмулируемой глобальной сети	13
2.12	обновление репозитория ПО	14
2.13	установка пакета для просмотра файлов png	14
2.14	подготовка каталога и создание необходимых файлов	15
2.15	создание скрипта для создания топологии сети	15
2.16	скрипт для визуализации результатов	16
2.17	Makefile	16
2.18	максимальная единица передачи	17
2.19	полученный график	17
2.20	редактирование файла	18
2.21	полученный график	18
2.22	скрипт для вычисления параметров на основе данных файла ping.dat	19
2.23	правило запуска скрипта	19
2.24	результат запуска	20
2.25	эксперимент по изменению задержки	20
2.26	график эксперимента по изменению задержки	21
2.27	вычисление параметров времени	21
2.28	джиттер	22
2.29	график	22

2.30 вычисление параметров времени	23
2.31 значения корреляции для джиттера и задержки	23
2.32 график	24
2.33 вычисление параметров времени	24
2.34 распределения времени задержки в эмулируемой глобальной сети	25
2.35 график	25
2.36 вычисление параметров времени	26

Список таблиц

1 Цель работы

Основной целью работы является знакомство с NETEM — инструментом для тестирования производительности приложений в виртуальной сети, а также получение навыков проведения интерактивного и воспроизводимого экспериментов по измерению задержки и её дрожания (jitter) в моделируемой сети в среде Mininet.

2 Выполнение лабораторной работы

2.1 Запуск лабораторной топологии

Запустите виртуальную среду с mininet. Из основной ОС подключимся к виртуальной машине. В виртуальной машине mininet при необходимости исправим права запуска X-соединения. Скопируйте значение куки (MIT magic cookie)¹ своего пользователя mininet в файл для пользователя root (рис. 2.1)

```
mininet@mininet-vm:~$ xauth list $DISPLAY
mininet-vm/unix:10 MIT-MAGIC-COOKIE-1 fa5520b19462a53b131d09eb09dd97c6
mininet@mininet-vm:~$ sudo -i
root@mininet-vm:~# xauth add mininet-vm/unix:10 MIT-MAGIC-COOKIE-1 fa5520b19462a53b131d09eb09dd97c6
root@mininet-vm:~# logout
mininet@mininet-vm:~$
```

Рис. 2.1: запуск виртуальной машины и исправление прав доступа

Зададим простейшую топологию, состоящую из двух хостов и коммутатора с назначенной по умолчанию mininet сетью 10.0.0.0/8. После введения этой команды запустятся терминалы двух хостов, коммутатора и контроллера. Терминалы коммутатора и контроллера можно закрыть (рис. 2.2)



Рис. 2.2: запуск графических окон

Проверим подключение между хостами h1 и h2 с помощью команды ping с параметром -c 6 (рис. 2.3)

min/avg/max/mdev = 0.052/0.569/2.820/1.010 ms

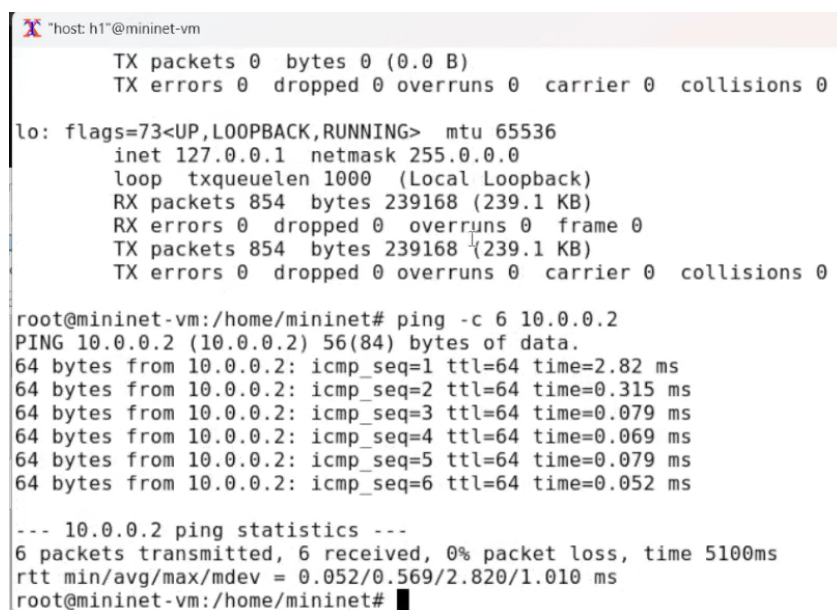


Рис. 2.3: проверка подключения между хостами

2.2 Интерактивные эксперименты

На хосте h1 добавим задержку в 100 мс к выходному интерфейсу. min/avg/max/mdev = 0.052/0.569/2.820/1.010 ms (рис. 2.4)


```

root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 100ms
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=101 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5008ms
rtt min/avg/max/mdev = 100.279/100.640/101.357/0.394 ms
root@mininet-vm:/home/mininet#

```

Рис. 2.4: проверка подключения соединения с задержкой 100 мс

Для эмуляции глобальной сети с двунаправленной задержкой необходимо к соответствующему интерфейсу на хосте h2 также добавить задержку в 100 миллисекунд. Проверим, что соединение между хостом h1 и хостом h2 имеет RTT в 200 мс (100 мс от хоста h1 к хосту h2 и 100 мс от хоста h2 к хосту h1), повторив команду ping с параметром -c 6 на терминале хоста h1 min/avg/max/mdev = 0.052/0.569/2.820/1.010 ms (рис. 2.5)

```

root@mininet-vm:/home/mininet# sudo tc qdisc add dev h2-eth0 root netem delay 100ms
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=202 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=200 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=201 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=200 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=201 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=200 ms

--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5009ms
rtt min/avg/max/mdev = 200.205/200.741/201.606/0.541 ms
root@mininet-vm:/home/mininet#

```

Рис. 2.5: проверка подключения соединения с задержкой 100 мс на хосте 2

2.3 Изменение задержки в эмулируемой глобальной сети

Изменим задержку со 100 мс до 50 мс для отправителя h1 и для получателя h2. min/avg/max/mdev = 0.052/0.569/2.820/1.010 ms (рис. 2.6)

```

root@mininet-vm:/home/mininet# sudo tc qdisc change dev h1-eth0 root netem delay 50ms
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=101 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5014ms
rtt min/avg/max/mdev = 100.381/101.175/102.471/0.733 ms
root@mininet-vm:/home/mininet# █

```

Рис. 2.6: Изменение задержки в эмулируемой глобальной сети со 100 мс до 50 мс

2.4 Восстановление исходных значений (удаление правил) задержки в эмулируемой глобальной сети

Восстановим конфигурацию по умолчанию, удалив все правила, применённые к сетевому планировщику соответствующего интерфейса для отправителя h1 и для получателя h2. Проверим, что соединение между хостом h1 и хостом h2 не имеет явно установленной задержки, используя команду ping с параметром -c 6 с терминала хоста h1. min/avg/max/mdev = 0.052/0.569/2.820/1.010 ms (рис. 2.7)

```

root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.68 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.447 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.178 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.073 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.069 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.114 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5100ms
rtt min/avg/max/mdev = 0.069/0.426/1.680/0.575 ms
root@mininet-vm:/home/mininet# █

```

Рис. 2.7: Восстановление исходных значений (удаление правил) задержки в эмулируемой глобальной сети

2.5 Добавление значения дрожания задержки в интерфейс подключения к эмулируемой глобальной сети

Добавим на узле h1 задержку в 100 мс со случайным отклонением 10 мс. Проверим, что соединение от хоста h1 к хосту h2 имеет задержку 100 мс со случайным отклонением ± 10 мс, используя в терминале хоста h1 команду ping с параметром -c 6. min/avg/max/mdev = 0.052/0.569/2.820/1.010 ms (рис. 2.8)

```
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 100ms 10ms
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=110 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=108 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=98.8 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=101 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5010ms
rtt min/avg/max/mdev = 98.773/103.294/110.451/4.270 ms
root@mininet-vm:/home/mininet#
```

Рис. 2.8: Добавление значения дрожания задержки в интерфейс подключения к эмулируемой глобальной сети

2.6 Добавление значения корреляции для джиттера и задержки в интерфейс подключения к эмулируемой глобальной сети

Добавим на интерфейсе хоста h1 задержку в 100 мс с вариацией ± 10 мс и значением корреляции в 25%. Убедимся, что все пакеты, покидающие устройство h1 на интерфейсе h1-eth0, будут иметь время задержки 100 мс со случайным отклонением ± 10 мс, при этом время передачи следующего пакета зависит от предыдущего значения на 25%. Используем для этого в терминале хоста h1 команду ping с параметром -c 20. min/avg/max/mdev =

0.052/0.569/2.820/1.010 ms (рис. 2.9) (рис. 2.10)

```
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 1
00ms 10ms 25%
root@mininet-vm:/home/mininet# ping -c 20 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=108 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=106 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=91.6 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=96.0 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=106 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=109 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=109 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=104 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=95.4 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=95.7 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=106 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=94.5 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=95.6 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=109 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=100 ms
```

Рис. 2.9: Добавление значения корреляции для джиттера и задержки в интерфейс подключения к эмулируемой глобальной сети

```
host: h1@mininet-vm
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=91.6 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=96.0 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=106 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=109 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=109 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=104 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=95.4 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=95.7 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=106 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=94.5 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=95.6 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=109 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=105 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=106 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19033ms
rtt min/avg/max/mdev = 91.632/102.285/109.397/5.474 ms
root@mininet-vm:/home/mininet#
```

Рис. 2.10: Добавление значения корреляции для джиттера и задержки в интерфейс подключения к эмулируемой глобальной сети

2.7 Распределение задержки в интерфейсе

подключения к эмулируемой глобальной сети

Зададим нормальное распределение задержки на узле h1 в эмулируемой сети. Убедимся, что все пакеты, покидающие хост h1 на интерфейсе h1-eth0, будут иметь время задержки, которое распределено в диапазоне 100 мс $\pm$ 20 мс. Используем для этого команду ping на терминале хоста h1 с параметром -c 10. (рис. 2.11)

```
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 1
00ms 20ms distribution normal
root@mininet-vm:/home/mininet# ping -c 10 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=116 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=82.1 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=108 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=105 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=78.7 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=131 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=118 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=122 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=71.8 ms

--- 10.0.0.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9025ms
rtt min/avg/max/mdev = 71.844/103.239/130.855/18.850 ms
root@mininet-vm:/home/mininet#
```

Рис. 2.11: Распределение задержки в интерфейсе подключения к эмулируемой глобальной сети

2.8 Воспроизведение экспериментов. Предварительная подготовка

Обновим репозитории программного обеспечения на виртуальной машине (рис. 2.12)

```

mininet@mininet-vm:~$ sudo apt-get update
Get:1 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
Hit:2 http://us.archive.ubuntu.com/ubuntu focal InRelease
Get:3 http://us.archive.ubuntu.com/ubuntu focal-updates InRelease [128 kB]
Get:4 http://us.archive.ubuntu.com/ubuntu focal-backports InRelease [128 kB]
Get:5 http://us.archive.ubuntu.com/ubuntu focal-updates/main i386 Packages [1,114 kB]
Get:6 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 Packages [3,955 kB]
Get:7 http://us.archive.ubuntu.com/ubuntu focal-updates/main Translation-en [600 kB]
Get:8 http://us.archive.ubuntu.com/ubuntu focal-updates/universe i386 Packages [823 kB]
Get:9 http://us.archive.ubuntu.com/ubuntu focal-updates/universe amd64 Packages [1,262 kB]
Get:10 http://us.archive.ubuntu.com/ubuntu focal-updates/universe Translation-en [303 kB]
Fetched 8,441 kB in 4s (1,987 kB/s)
Reading package lists... Done
mininet@mininet-vm:~$

```

Рис. 2.12: обновление репозитория ПО

Установим пакет `gexie` — понадобится для просмотра файлов `png` (рис. 2.13)

```

Setting up gnome-control-center (1:3.36.5-0ubuntu4.1) ...
Processing triggers for mime-support (3.64ubuntu1) ...
Processing triggers for hicolor-icon-theme (0.17-2) ...
Processing triggers for libgtk-3-0:amd64 (3.24.20-0ubuntu1) ...
Processing triggers for libgtk2.0-0:amd64 (2.24.32-4ubuntu4.1) ...
Processing triggers for libc-bin (2.31-0ubuntu9) ...
Processing triggers for systemd (245.4-4ubuntu3.4) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for gnome-icon-theme (3.12.0-3) ...
Processing triggers for shared-mime-info (1.15-1) ...
Processing triggers for udev (245.4-4ubuntu3.4) ...
Processing triggers for sgml-base (1.29.1) ...
Setting up sgml-data (2.0.11) ...
Processing triggers for sgml-base (1.29.1) ...
Setting up docbook-xml (4.5-9) ...
Processing triggers for dbus (1.12.16-2ubuntu2.3) ...
Processing triggers for rygel (0.38.3-1ubuntu1) ...
Processing triggers for sgml-base (1.29.1) ...
mininet@mininet-vm:~$

```

Рис. 2.13: установка пакета для просмотра файлов `png`

2.9 Добавление задержки для интерфейса, подключающегося к эмулируемой глобальной сети

С помощью API Mininet воспроизведите эксперимент по добавлению задержки для интерфейса хоста, подключающегося к эмулируемой глобальной сети. В виртуальной среде mininet в своём рабочем каталоге с

проектами создадим каталог simple-delay и перейдем в него (рис. 2.14)
(рис. 2.15)

```
mininet@mininet-vm:~/work$ cd ..
mininet@mininet-vm:~$ mkdir -p ~/work/lab_netem_i/simple-delay
mininet@mininet-vm:~$ ls ~/work/lab_netem_i/simple-delay
mininet@mininet-vm:~$ cd ~/work/lab_netem_i/simple-delay
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$ mcedit lab_netem_i.py
```

Рис. 2.14: подготовка каталога и создание необходимых файлов

Создадим скрипт для эксперимента lab_netem_i.py (рис. 2.15)

```
/home/mininet i.py [-M--] 14 L:[ 1+ 0 1/ 49] *(14 /1229b) 32 0x020 [*][X]
#!/usr/bin/env python

"""
simple experiment.
Output: ping.dat
"""

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info
import time

def emptyNet():
    """Create an empty network and add nodes to it"""
    net = Mininet( controller=Controller, waitConnected=True )

    info( "*** Adding controller\n" )
    net.addController( 'c0' )

    info( "*** Adding hosts\n" )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )

    info( "*** Adding switch\n" )
    s1 = net.addSwitch( 's1' )

    info( "*** Starting network\n" )

1Help 2Save 3Mark 4Replac 5Copy 6Move 7Search 8Delete 9PullDn 10Quit
```

Рис. 2.15: создание скрипта для создания топологии сети

Создим скрипт для визуализации ping_plot результатов эксперимента
(рис. 2.16)

```

/home/mi~ing_plot [----] 26 L:[ 1+ 0
#!/usr/bin/gnuplot --persist

set terminal png crop
set output 'ping.png'
set xlabel "Sequence number"
set ylabel "Delay (ms)"
set grid
plot "ping.dat" with lines

```

Рис. 2.16: скрипт для визуализации результатов

Создадим Makefile для управления процессом проведения эксперимента (рис. 2.17)

```

/home/mi~Makefile [-M--] 26 L:[ 1+10 11/ 1
all: ping.dat ping.png

ping.dat:
<----->sudo python lab_netem_i.py
<----->sudo chown mininet:mininet ping.dat

ping.png: ping.dat
<----->./ping_plot

clean:
<----->-rm -f *.dat *.png

```

Рис. 2.17: Makefile

Выполним эксперимент (рис. 2.18) (рис. 2.19)


```

***Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Set delay
*** h1 : ('tc qdisc add dev h1-eth0 root netem delay 100ms',)
*** h2 : ('tc qdisc add dev h2-eth0 root netem delay 100ms',)
*** Ping
*** h1 : ('ping -c 100', '10.0.0.2', '| grep "time=" | awk \'{print $5, $7}\'' |
sed -e \'/time=//g\' -e \'/icmp_seq=//g\' > ping.dat')
*** Stopping network*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
sudo chown mininet:mininet ping.dat
./ping_plot

```

Рис. 2.18: максимальная единица передачи

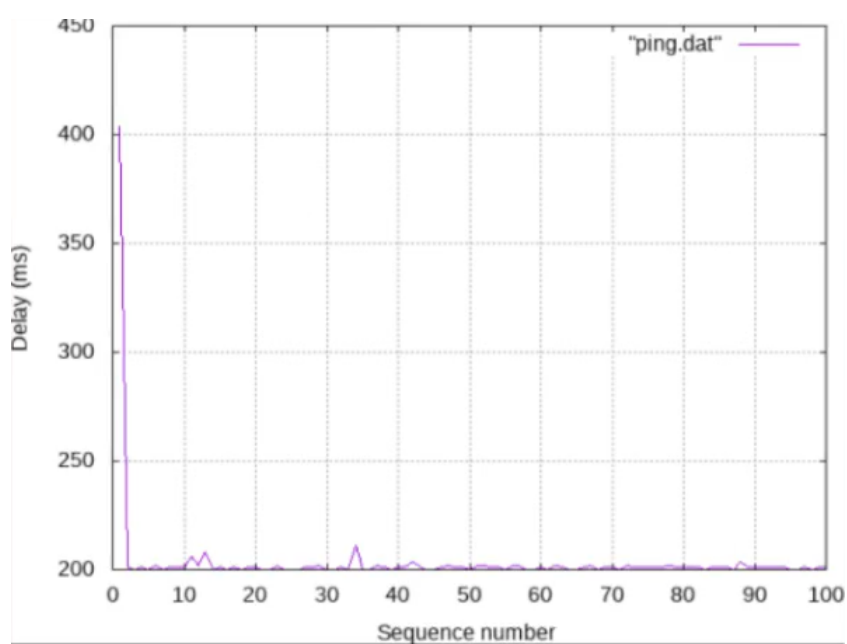


Рис. 2.19: полученный график

Из файла ping.dat удалим первую строку и заново построим график (рис. 2.20) (рис. 2.21)

```
/home/mi~ping.dat [-M--] 0 L:[ 1+ 0 1/100] *(0 / 686b) 50 0
2 201
3 200
4 201
5 200
6 202
7 200
8 201
9 201
10 201
11 206
12 202
13 208
14 200
15 201
16 200
17 201
18 200
19 201
20 201
```

Рис. 2.20: редактирование файла

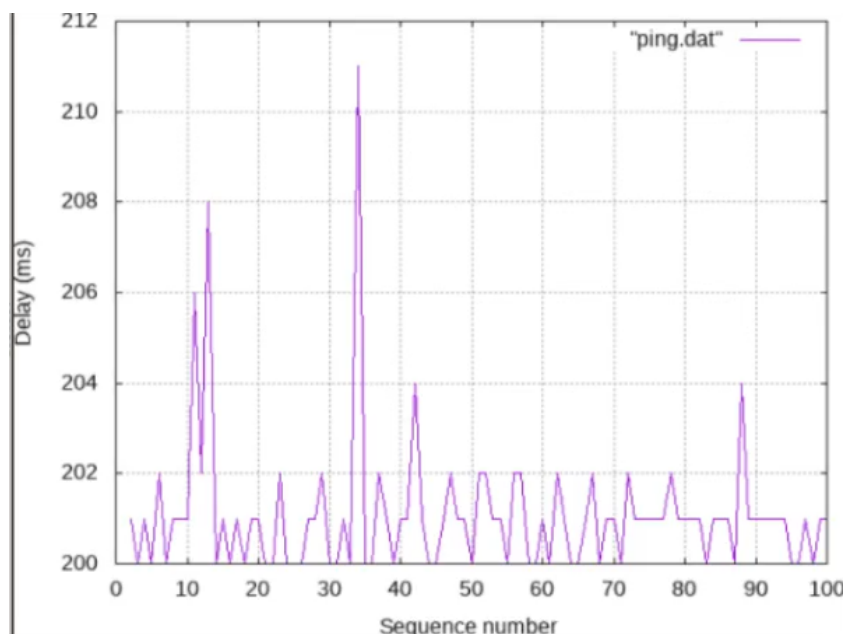


Рис. 2.21: полученный график

Разработаем скрипт для вычисления на основе данных файла ping.dat минимального, среднего, максимального и стандартного отклонения времени приёма-передачи (рис. 2.22)

```

/home/mi~y/rtt.py  [-M--] 15 L:[ 1+23 24/ 24] *(571 / 571b) <EC
import numpy as np

def calc_times(data):
    times = np.array
    min_time = np.min(times)
    avg_time = np.mean(times)
    max_time = np.max(times)
    std_dev = np.std(times)

    return min_time, avg_time, max_time, std_dev

def read_file():
    with open('ping.dat', 'r') as file
    data = file.readlines()

    min_time, avg_time, max_time, std_dev = calc_times(data)

    print(f'Min time: {min_time} ms')
    print(f'Avg time: {avg_time} ms')
    print(f'Max time: {max_time} ms')
    print(f'Std dev: {std_dev} ms')

if __name__ == "__main__":
    read_file()

```

Рис. 2.22: скрипт для вычисления параметров на основе данных файла ping.dat

Добавьте правило запуска скрипта в Makefile (рис. 2.23)

```

/home/mi~Makefile  [-M--] 21 L:[ 1+10 11/ 11]
all: ping.dat ping.png

ping.dat:
<----->sudo python lab_netem_i.py
<----->sudo chown mininet:mininet ping.dat

ping.png: ping.dat
<----->./ping_plot

stats: ping.dat
<----->python rtt.py

clean:
<----->-rm -f *.dat *.png

```

Рис. 2.23: правило запуска скрипта

Продemonстрируем работу скрипта с выводом значений на экран или в отдельный файл (рис. 2.24)

```
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$ make stats
python rtt.py
Min time: 200.0 ms
Avg time: 201.0909090909091 ms
Max time: 211.0 ms
Std dev: 1.5510889566145125 ms
```

Рис. 2.24: результат запуска

2.10 Задание для самостоятельной работы

Реализуем евоспроизводимые эксперименты по изменению задержки, джиттера, значения корреляции для джиттера и задержки, распределения времени задержки в эмулируемой глобальной сети. Постройте графики. Вычислите минимальное, среднее, максимальное и стандартное отклонение времени приёма-передачи для каждого случая

Эксперимент по изменению задержки (рис. 2.25) (рис. 2.26) (рис. 2.27)

```
/home/mi~tem i.py [B---] 0 L: [ 22+16 38/ 49] *(920 /1227b) 10 0x00A [*][X]
h1 = net.addHost( 'h1', ip='10.0.0.1' )
h2 = net.addHost( 'h2', ip='10.0.0.2' )

info( '***adding switch\n' )
s1 = net.addSwitch( 's1' )

info( '***creating links\n' )
net.addLink( h1, s1 )
net.addLink( h2, s1 ) ...

info( '***starting network\n' )
net.start()

info( '***set delay\n' )
h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem delay 50ms' )
h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem delay 50ms' )

time.sleep(10)

info( '***Ping\n' )
h1.cmdPrint( 'ping -c 100', h2.IP(), '[ -q -p "times" ] && \'\(print $?, $?)\'

info( '***stopping network\n' )
net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()
```

Рис. 2.25: эксперимент по изменению задержки

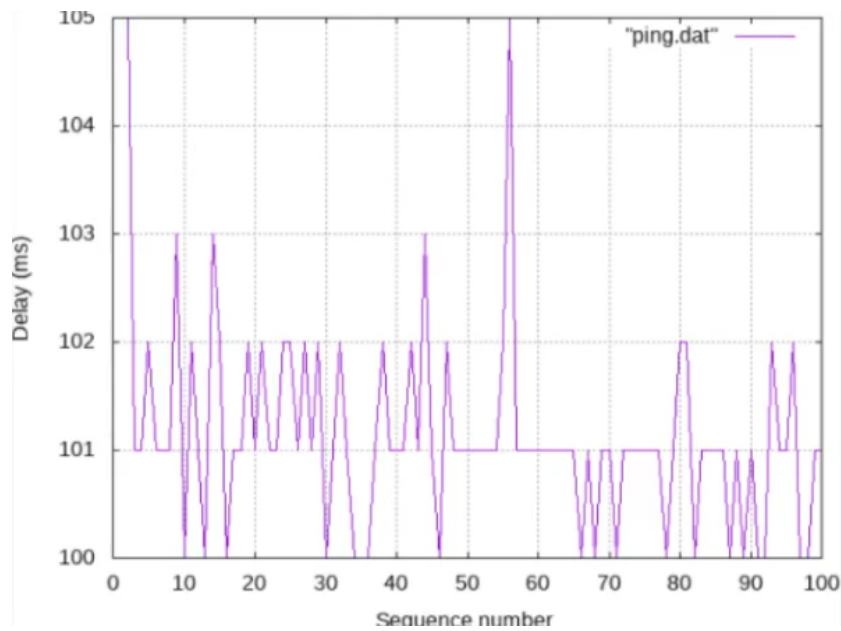


Рис. 2.26: график эксперимента по изменению задержки

```
mininet@mininet-vm:~/work/lab_netem_i/change-delay$ make stats
python rtt.py
Min time: 100.0 ms
Avg time: 101.13131313131314 ms
Max time: 105.0 ms
Std dev: 0.8949517750843105 ms
mininet@mininet-vm:~/work/lab_netem_i/change-delay$
```

Рис. 2.27: вычисление параметров времени

Добавление значения для джиттера и задержки в интерфейс подключения к эмулируемой глобальной сети (рис. 2.28) (рис. 2.29) (рис. 2.30)

```

/home/mi~tem i.py [BM--] 68 L: [ 22+15 37/ 49] *(926 /1234b) 10 0x00A [*][X]
h1 = net.addHost( 'h1', ip= '30.0.0.1' )
h2 = net.addHost( 'h2', ip= '30.0.0.2' )

info( '***Adding switch\n' )
s1 = net.addSwitch( 's1' )

info( '***Creating links\n' )
net.addLink( h1, s1 )
net.addLink( h2, s1 )....

info( '*** Starting network\n' )
net.start()

info( '*** Set delay\n' )
h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem delay 100ms 10ms' )
h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem delay 100ms' )

time.sleep(10)

info( '*** Ping\n' )
h1.cmdPrint( 'ping -c 100', h2.IP(), '[ -q -p "time=" ] >>> \'\{print "%5.2f\'' )

info( '*** Stopping network' )
net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()
1Help 2Save 3Mark 4Replac 5Copy 6Move 7Search 8Delete 9PullDn10Quit

```

Рис. 2.28: джиттер

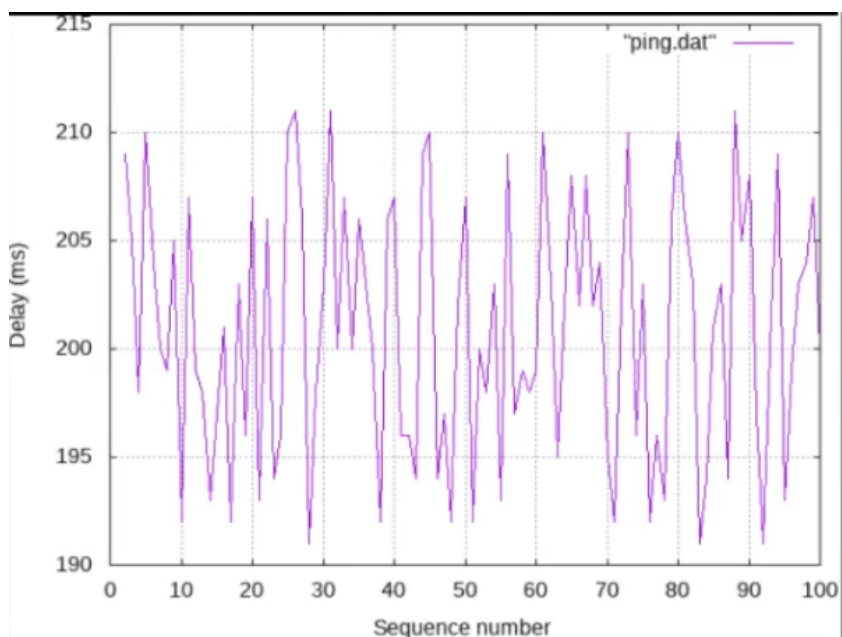


Рис. 2.29: график

```
mininet@mininet-vm:~/work/lab_netem_i/jitter-delay$ make stats
python rtt.py
Min time: 191.0 ms
Avg time: 200.88888888888889 ms
Max time: 211.0 ms
Std dev: 5.911619937088616 ms
mininet@mininet-vm:~/work/lab_netem_i/jitter-delay$
```

Рис. 2.30: вычисление параметров времени

Значения корреляции для джиттера и задержки (рис. 2.31) (рис. 2.32) (рис. 2.33)

```
/home/minitem i.py [BM--] 68 L:[ 22+15 37/ 49] *(930 /1238b) 10 0x00A [*][X]
h1 = net.addHost( 'h1', ip= '10.0.0.1' )
h2 = net.addHost( 'h2', ip= '10.0.0.2' )

info( '*** Adding switch\n' )
s1 = net.addSwitch( 's1' )

info( '*** Creating links\n' )
net.addLink( h1, s1 )
net.addLink( h2, s1 ) ...

info( '*** Starting network\n' )
net.start()

info( '*** Set delay\n' )
h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem delay 100ms 10ms 25%' )
h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem delay 100ms' )

time.sleep(10)

info( '*** Ping\n' )
h1.cmdPrint( 'ping -c 100', h2.IP(), '| grep "time" | awk \'{print $5, $7}\'' )

info( '*** Stopping network' )
net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()
```

Рис. 2.31: значения корреляции для джиттера и задержки

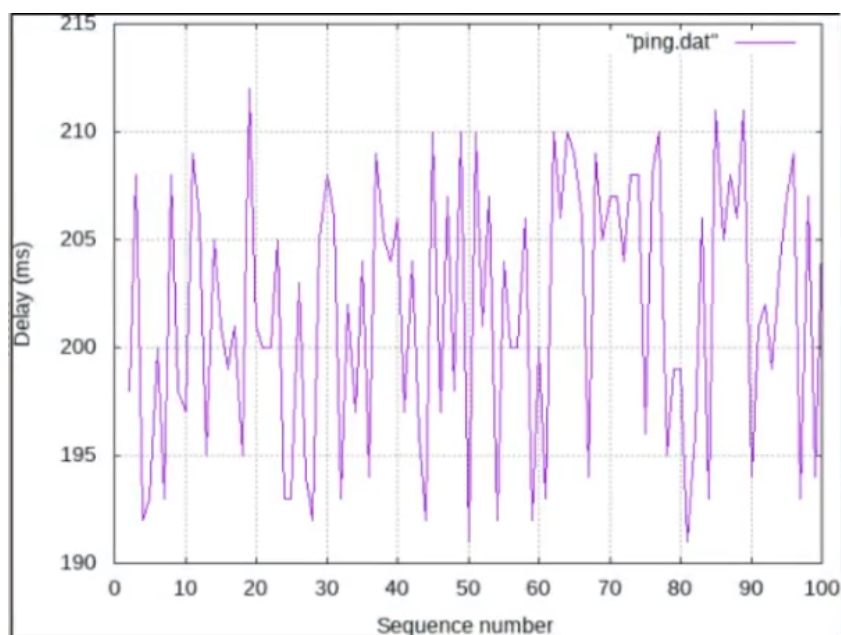


Рис. 2.32: график

```
mininet@mininet-vm:~/work/lab_netem_i/correlation-delay$ make stats
python rtt.py
Min time: 191.0 ms
Avg time: 201.64646464646464 ms
Max time: 212.0 ms
Std dev: 6.0924081442583 ms
mininet@mininet-vm:~/work/lab_netem_i/correlation-delay$
```

Рис. 2.33: вычисление параметров времени

Распределения времени задержки в эмулируемой глобальной сети (рис. 2.34) (рис. 2.35) (рис. 2.36)


```
mininet@mininet-vm:~/work/lab_netem_i/distribution-delay$ make stats
python rtt.py
Min time: 147.0 ms
Avg time: 200.4949494949495 ms
Max time: 252.0 ms
Std dev: 19.004716978205483 ms
mininet@mininet-vm:~/work/lab_netem_i/distribution-delay$
```

Рис. 2.36: вычисление параметров времени

3 Выводы

В результате выполнения лабораторной работы было проведено знакомство с NETEM — инструментом для тестирования производительности приложений в виртуальной сети, а также получение навыков проведения интерактивного и воспроизводимого экспериментов по измерению задержки и её дрожания (jitter) в моделируемой сети в среде Mininet.